

Modelos de Inteligencia Artificial

Práctica 1.1

Algoritmos Genéticos



Curso de Especialización de Inteligencia Artificial y Big Data

Curso 2025-2026

IES Camas-Antonio Brisquet

Índice

Índice	2
¿En qué consiste la práctica?	2
Objetivos de la práctica	2
Ficheros proporcionados	3
altitud.asc	3
cargar_datos.py	4
elevaciones.py	5
altitud_maxima.py	8

¿En qué consiste la práctica?

La siguiente práctica consiste en resolver un problema de optimización de altitudes en la zona de los Pirineos. Para ello, se usará un algoritmo genético en Python (PyGad).

El mapa está representado como un array bidimensional donde cada posición contiene la altitud en metros.

Objetivos de la práctica

Los objetivos de esta práctica es usar el algoritmo genético de PyGad para generar candidatos en formato de pares (x,y) dos puntos que representan los de mayor y menor altitud.

cargar_datos.py

En este fichero codificamos la función que se encargará de leer el archivo ASCII anteriormente descrito. Esta función se encarga de almacenar el número de columnas, filas y el valor de dato inválido (nodata), así como reemplazar caracteres para optimizar la lectura.

```
cargar_datos.py > leer
1  import numpy as np
2
3  def leer(nombrefichero: str) :
4      with open(nombrefichero) as fichero:
5          linea = fichero.readline()
6          ncols = int(linea.split()[1])
7          linea = fichero.readline()
8          nrows = int(linea.split()[1])
9          array = np.ndarray((ncols, nrows), float)
10         print(array.shape)
11         fichero.readline()
12         fichero.readline()
13         fichero.readline()
14         nodata = int(fichero.readline().split()[1])
15         print('nodata:', nodata)
16         for row in range(nrows):
17             linea = fichero.readline()
18             col = 0
19             values = linea.split(' ')
20             for col in range(ncols):
21                 value = float(values[col].replace(',', '.'))
22                 array[col][row] = value
23                 if value < -100:
24                     array[col][row] = -2000
25     return array
26
```

[elevaciones.py](#)

En este fichero son varias las funciones que podemos encontrar. Destacamos dos de ellas.

En la función `obtener_mapa()` hacemos una carga del mapa usando la función `leer` del fichero `cargar_datos.py` pero con una peculiaridad: sólo nos interesa dos rangos de coordenadas, que son 8000-10000 para el eje x, y de 0-1000 para el eje y.

Con esta delimitación obtenemos el fragmento del mapa que representa solo a los Pirineos.

```
def obtener_mapa():  
    # Cargamos el mapa y nos quedamos con la parte que nos interesa  
    mapa = leer('altitud.asc')[8000:10000,0:1000]  
    print('----- Fichero Cargado -----')  
    print(mapa.shape)  
    return mapa
```

Por otro lado, tenemos la función `mostrar_mapa(mapa, punto_max, punto_min)`. Esta función se encarga de representar el mapa. Los parámetros `punto_max` y `punto_min` son opcionales, por lo que solo representa estos dos puntos en el caso de que se obtengan y se le pase por parámetro.

Al punto mínimo se le ha realizado varios ajustes como mostrar un pilar que lo une con el suelo ya que al representarlo directamente en el suelo, el punto queda embebido por el relieve del mapa y no se visualiza, por ello se le suma 2000 puntos de altitud a su altitud real.

```
def mostrar_mapa(mapa, punto_max=None, punto_min=None):
    fig, ax = plt.subplots(subplot_kw={"projection": "3d"})
    X = np.arange(mapa.shape[1])
    Y = np.arange(mapa.shape[0])
    X, Y = np.meshgrid(X, Y)

    surf = ax.plot_surface(X, Y, mapa, cmap=cm.terrain, linewidth=0, antialiased=False)

    # Customize the z axis.
    ax.set_zlim(-1.01, 2000)
    ax.zaxis.set_major_locator(LinearLocator(10))
    ax.zaxis.set_major_formatter('{x:.02f}')

    # Add a color bar which maps values to colors.
    fig.colorbar(surf, shrink=0.5, aspect=5)

    if punto_max: #Punto máximo
        ax.scatter(punto_max[1], punto_max[0], mapa[punto_max[0], punto_max[1]],
                  s=150, c='red', marker='o', label='Máximo', edgecolors="black")

    if punto_min: # Punto mínimo
        x = punto_min[1]
        y = punto_min[0]
        z_real = mapa[y, x]          # altitud real del punto
        z_top = z_real + 2000        # elevamos el punto para que sea visible

        # — Pilar vertical —
        ax.plot([x, x], [y, y], [z_real, z_top], color='purple', linewidth=4)

        # — Punto en la cima del pilar —
        ax.scatter(x, y, z_top, s=150, c='purple', marker='o', label='Mínimo', edgecolors="black")

    plt.legend()
    plt.show()
```

altitud_maxima.py

En este fichero es donde realizamos la búsqueda de los puntos máximos y mínimos mediante algoritmo genético. Para ello definimos dos funciones fitness que son las que utilizaremos.

```
mapa = elev.obtener_mapa()
# Vamos a maximizar la altitud para las coordenadas x, y

def fitness_func(ga_instance, solution, solution_idx): # Optimiza la máxima altitud
    maxX, maxY = mapa.shape
    x = int(solution[0])
    y = int(solution[1])
    if x >= maxX or y >= maxY:
        return -1000
    return mapa[x][y]

def fitness_func_minima(ga_instance, solution, solution_idx):
    # Esta función optimiza la mínima altitud
    maxX, maxY = mapa.shape
    x = int(solution[0])
    y = int(solution[1])
    # Si sale de los límites o si está en el mar
    if x >= maxX or y >= maxY or x < 0 or y < 0 or mapa[x][y] <= 0:
        return -1000
    return 1 / (mapa[x][y]+1e-6) #Le sumamos un valor pequeño para evitar división por cero
```

Casos de prueba

A continuación realizaremos varias ejecuciones antes y después de ajustar los parámetros para ver cómo varía la exactitud del programa.

Ejecuciones antes de ajustar parámetros

Mostramos los parámetros predeterminados:

```
num_generations = 25
num_parents_mating = 40

sol_per_pop = 2000
num_genes = 2
mutation_num_genes=2

init_range_low = [0,0]
init_range_high = [2000,1000]

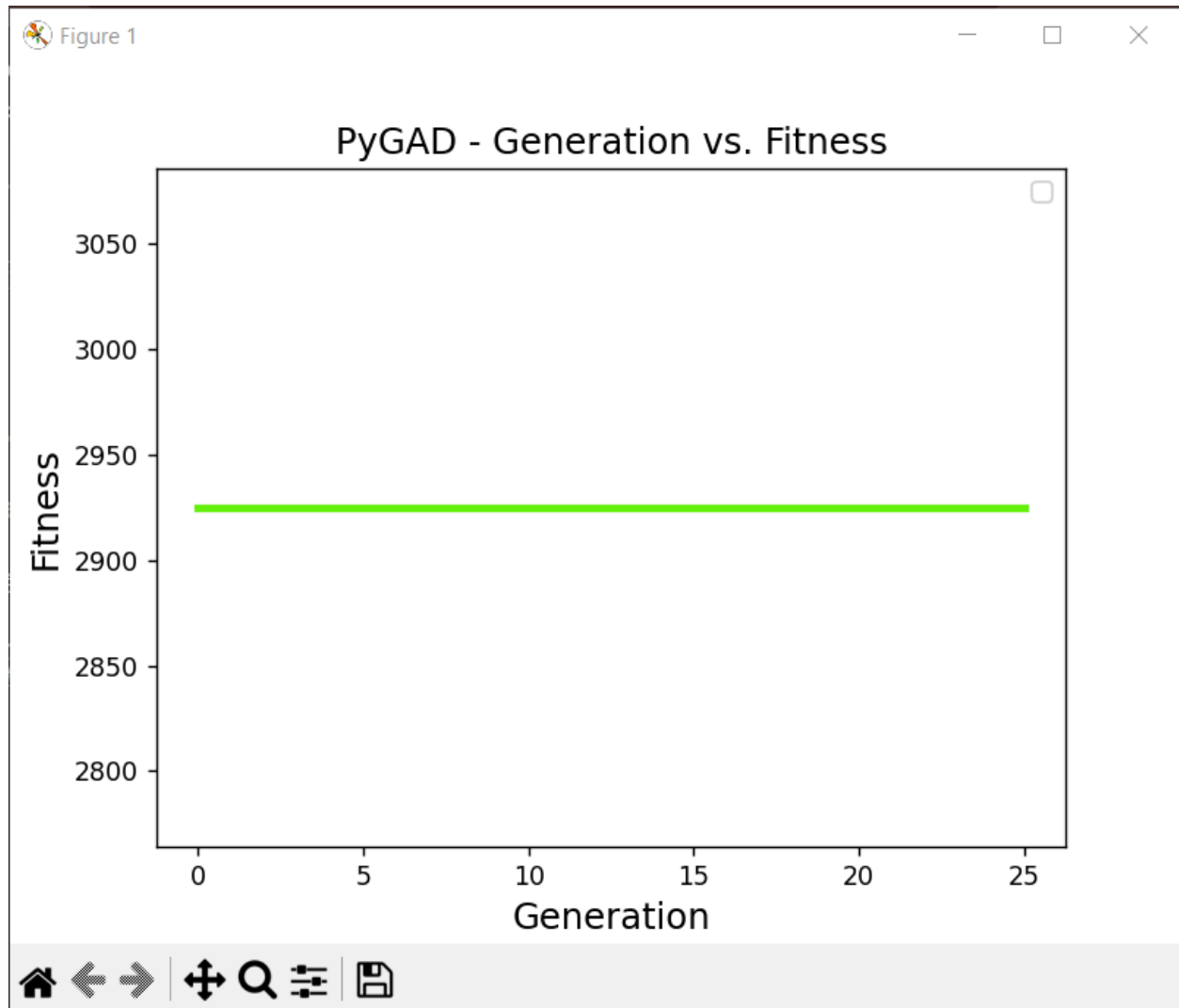
parent_selection_type = "sss"
keep_parents = 1

crossover_type = "two_points"

mutation_type = "random"
mutation_percent_genes = 30
gene_type = int
```

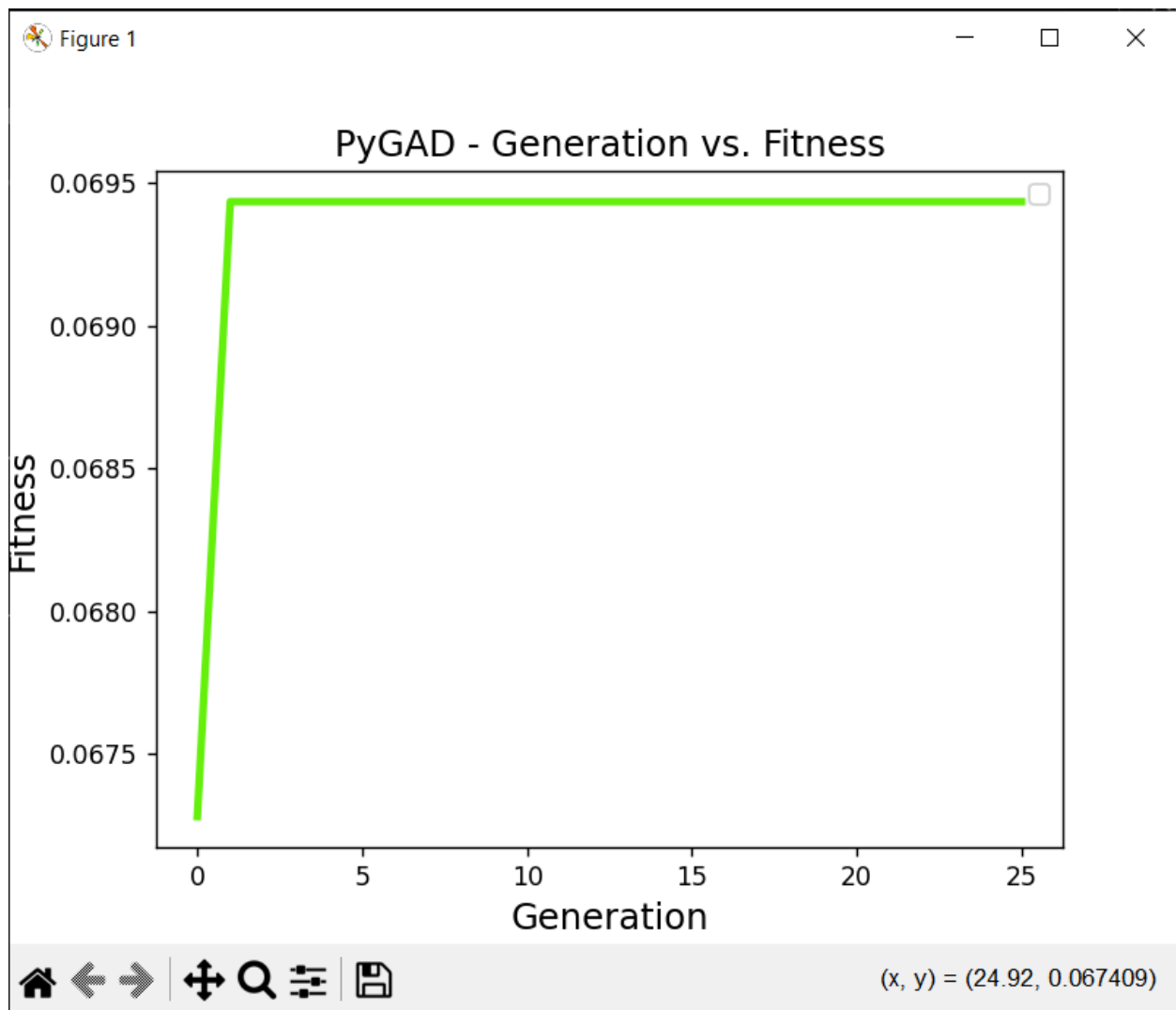
Ejecución 1

Máximo:

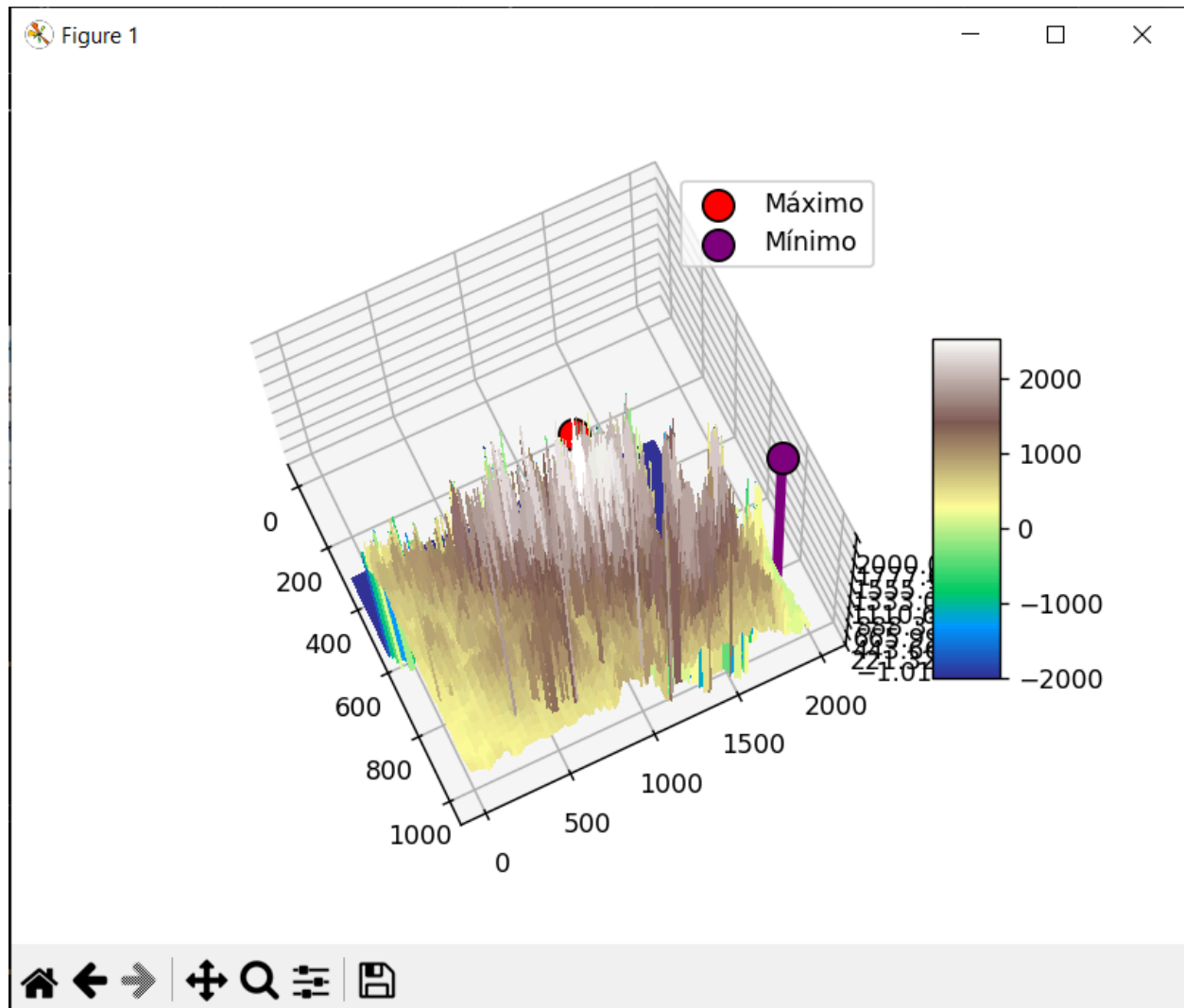


```
Parameters of the best solution : [1011 665]
Fitness value of the best solution = 2924.945
C:\Users\David\Downloads\Ficheros para la Práctica 1.1-2020\
packages\pygad\visualize\plot.py:120: UserWarning: No artists
d to put in legend. Note that artists whose label starts with
e ignored when legend() is called with no argument.
  matplotlib.legend()
Predicted output based on the best solution : 2924.945
Valor de altitud maximo encontrado: 2924.945
```

Mínimo:

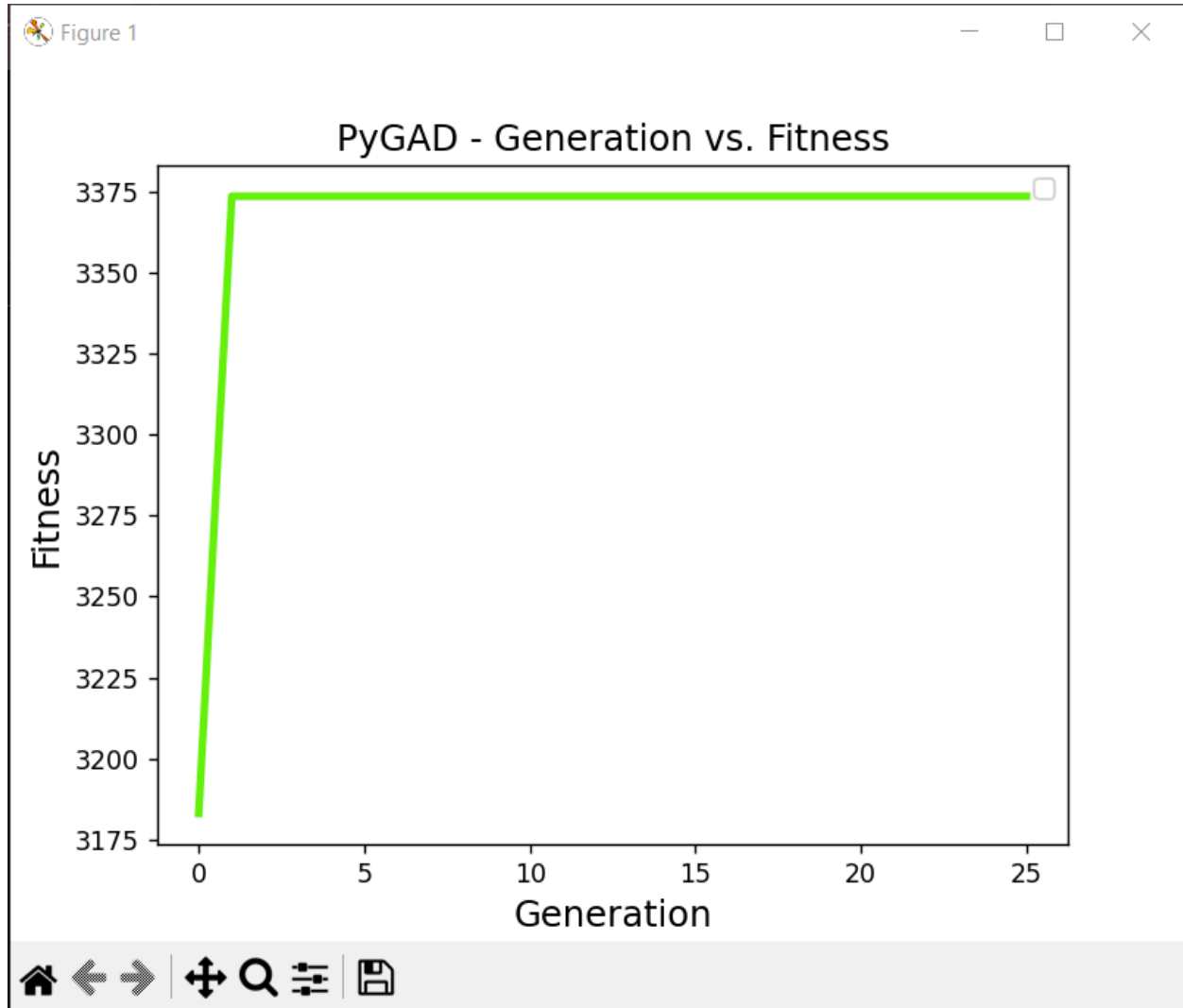


```
Parameters of the best solution : [1999  805]
Fitness value of the best solution = 0.06720429655885103
Predicted output based on the best solution : 14.88
Valor de altitud minimo encontrado: 14.88
```



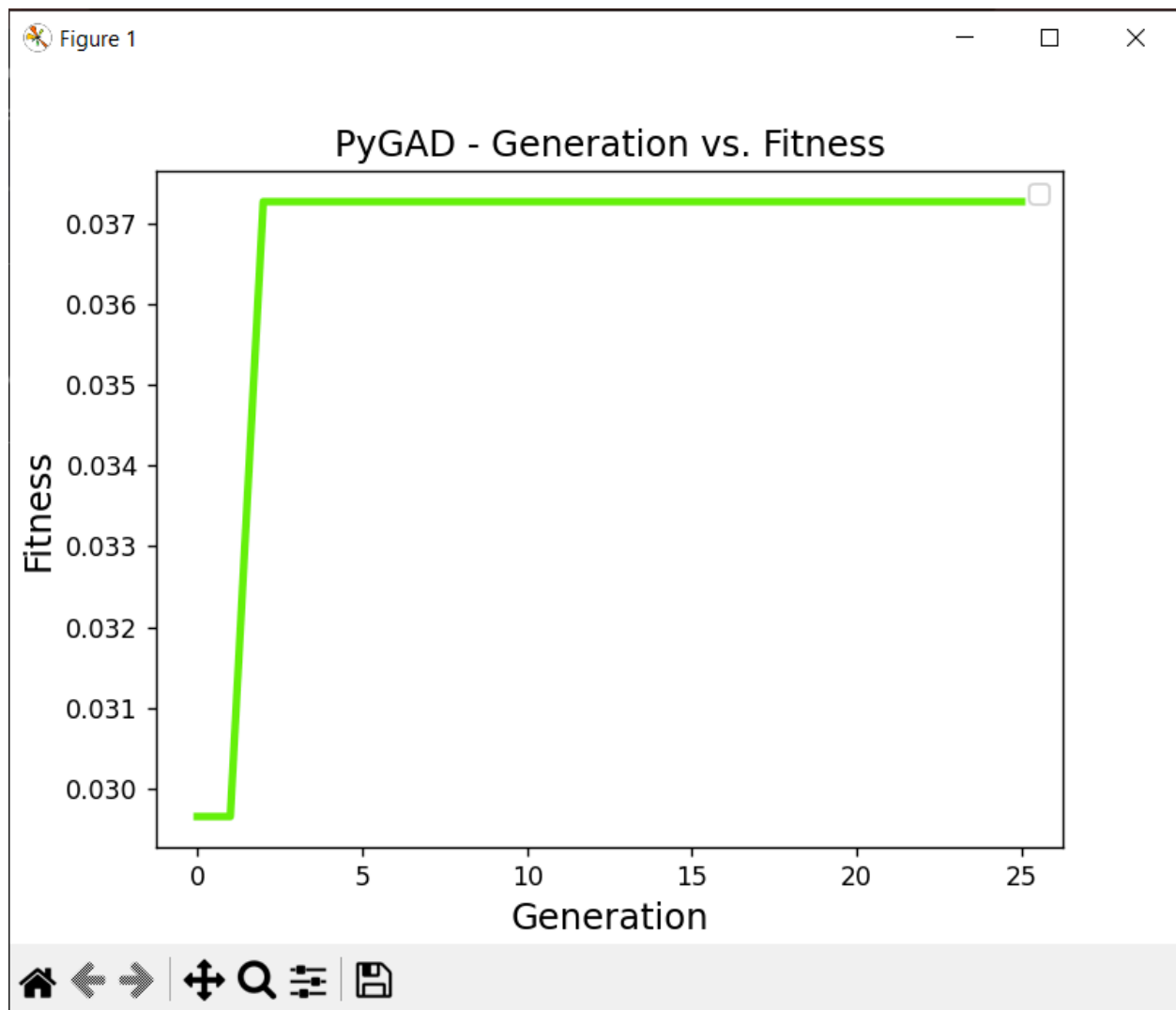
Ejecución 2

Máxima

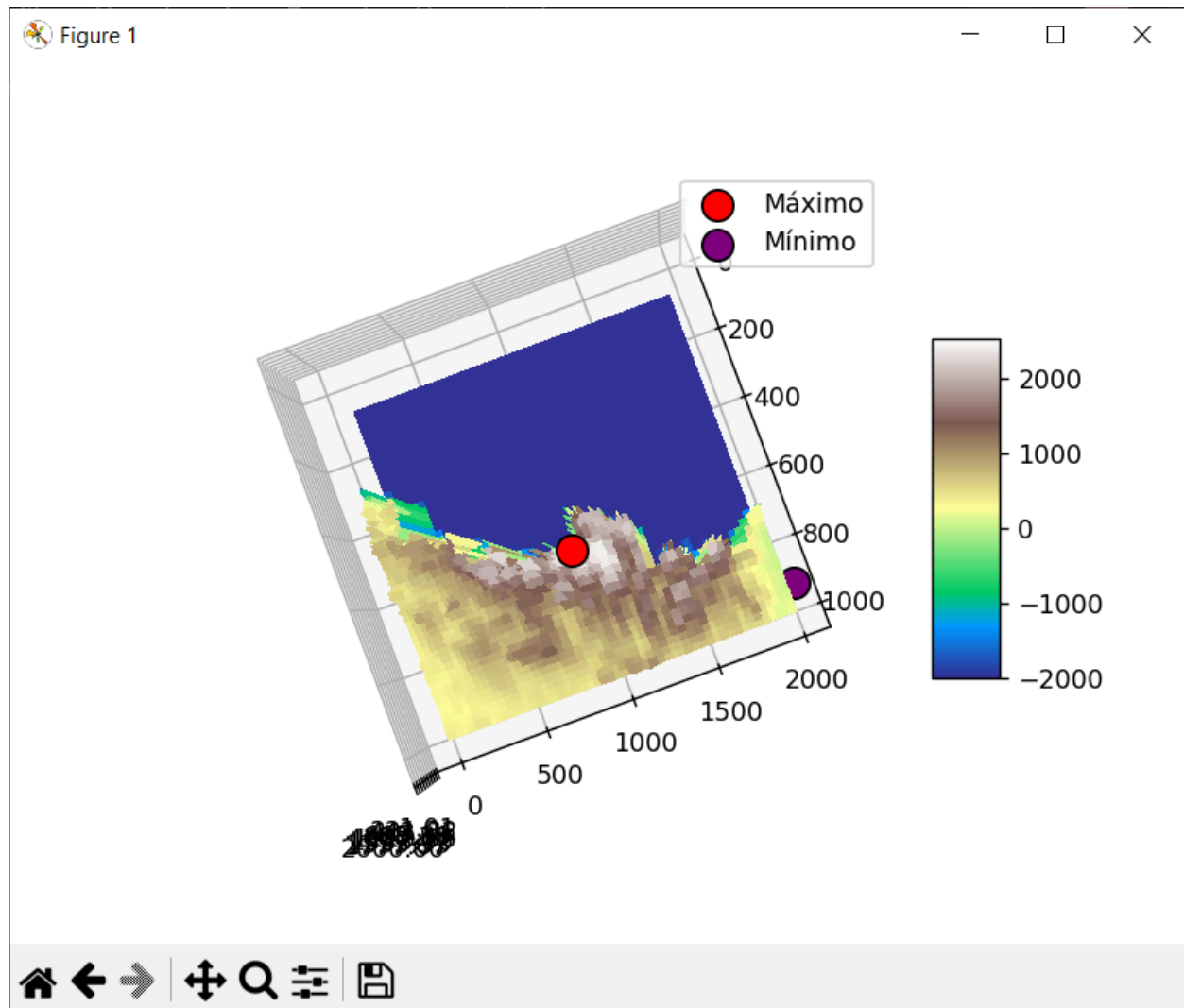


```
Parameters of the best solution : [1021 664]
Fitness value of the best solution = 3285.983
C:\Users\David\Downloads\Ficheros para la Práctica 1.1-2
packages\pygad\visualize\plot.py:120: UserWarning: No ar
d to put in legend. Note that artists whose label start
e ignored when legend() is called with no argument.
  matplotlib.legend()
Predicted output based on the best solution : 3285.983
Valor de altitud maximo encontrado: 3285.983
```

Mínima

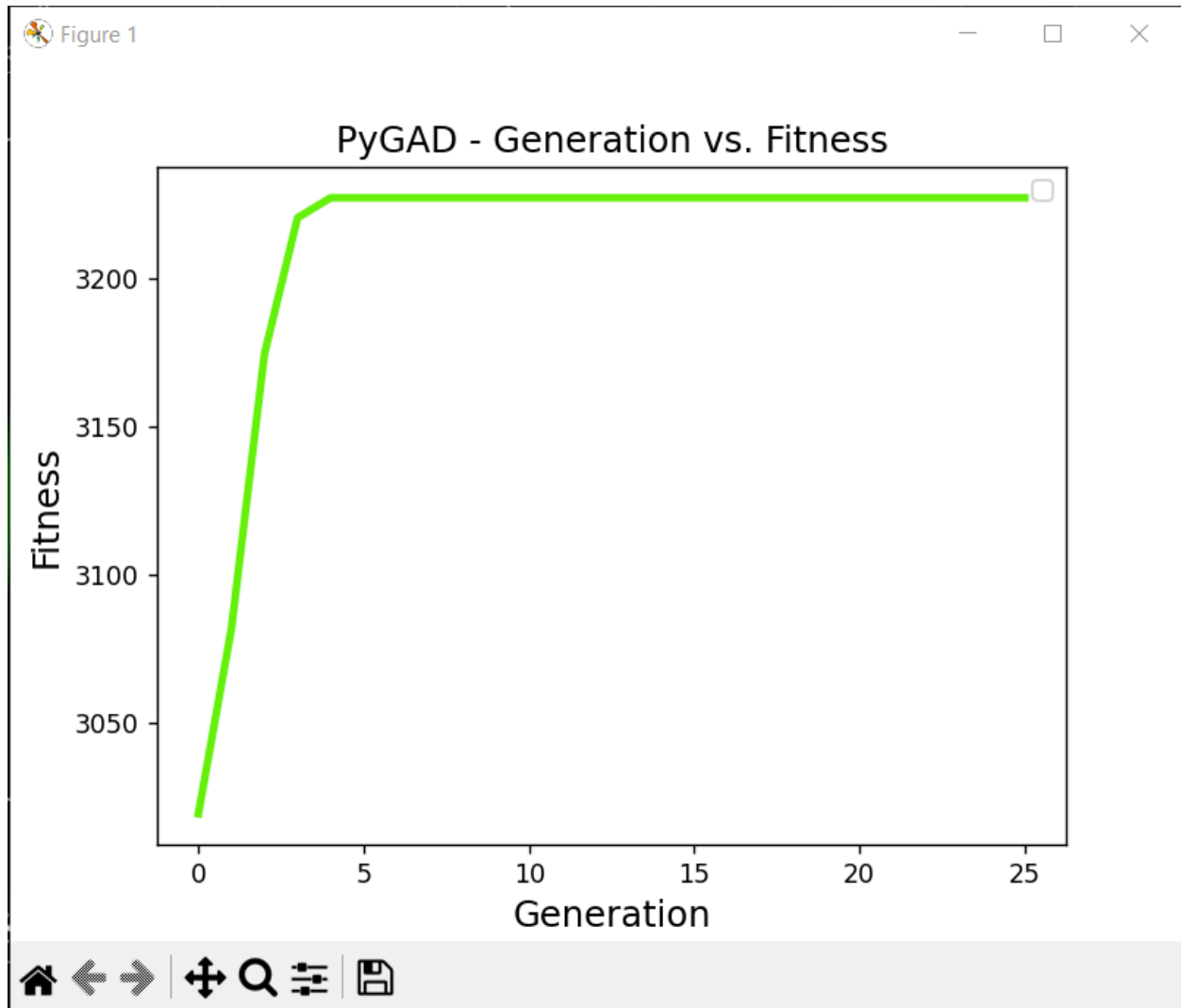


```
=====  
Parameters of the best solution : [1988  919]  
Fitness value of the best solution = 0.037223151415479196  
Predicted output based on the best solution : 26.865  
Valor de altitud minimo encontrado: 26.865  
□
```



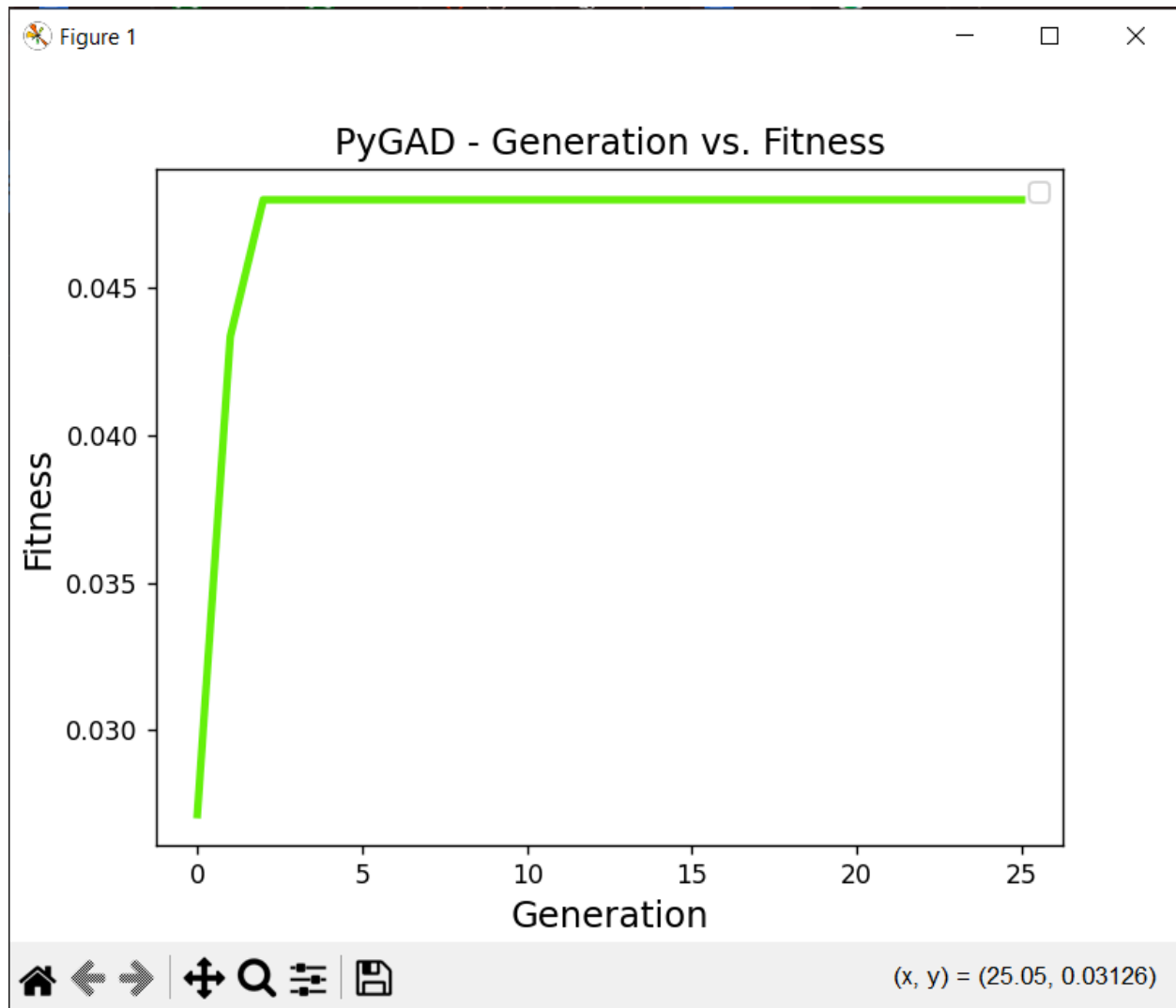
Ejecución 3

Máxima

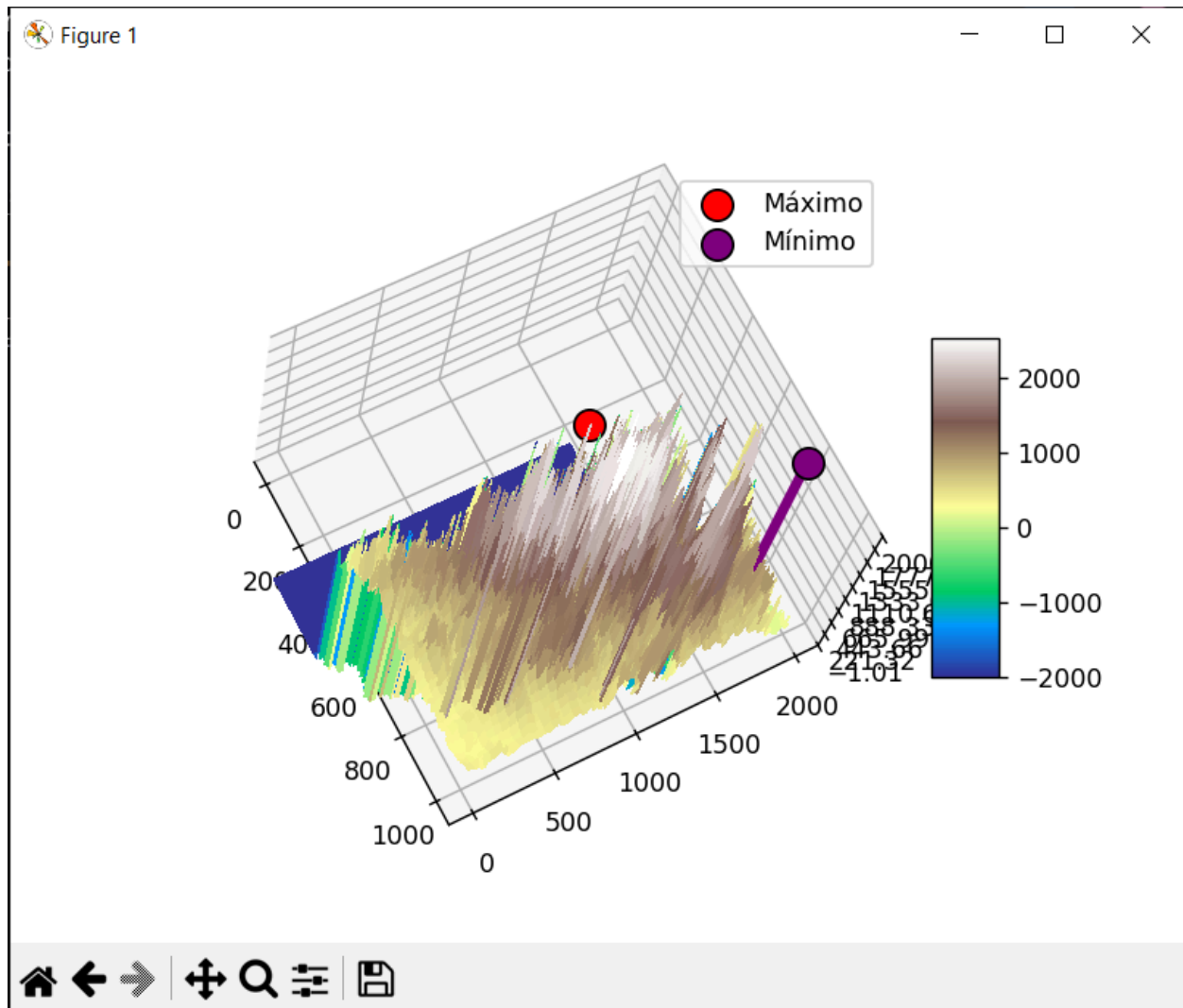


```
Parameters of the best solution : [756 642]
Fitness value of the best solution = -2000.0
C:\Users\David\Downloads\Ficheros para la Práctica 1.1-20
packages\pygad\visualize\plot.py:120: UserWarning: No art
d to put in legend. Note that artists whose label start
e ignored when legend() is called with no argument.
  matplotlib.legend()
Predicted output based on the best solution : 3227.227
Valor de altitud maximo encontrado: 3227.227
```


Mínima.



```
Parameters of the best solution : [1989  817]
Fitness value of the best solution = 0.047973132742953575
Predicted output based on the best solution : 20.845
Valor de altitud minimo encontrado:  20.845
```



Conclusiones

Tras corregir la acotación del mapa, el algoritmo genético ha comenzado a identificar valores de altitud mucho más realistas para los Pirineos, alcanzando picos entre **2924 m** y **3285 m**. Estos resultados se aproximan a las elevaciones máximas reales del macizo de Maladeta, donde se encuentra el Aneto, lo que confirma que el algoritmo converge correctamente hacia las zonas más altas del mapa. Las variaciones respecto a los 3404 m reales se deben a las limitaciones y suavizado del modelo digital de elevación utilizado, no al funcionamiento del algoritmo. En conjunto, los resultados muestran que el sistema es capaz de localizar de forma consistente las áreas más elevadas y cumplir eficazmente el objetivo marcado.

Ejecuciones después de ajustar parámetros

Ajuste de parámetros

Realizamos unos pequeños ajustes para comprobar cómo varían los resultados

```
num_generations = 300 #Antes 25. Explora espacio de bussqueda
num_parents_mating = 40 #Antes 40

sol_per_pop = 3500 #Antes 2000
num_genes = 2
mutation_num_genes=1

init_range_low = [0,0]
init_range_high = [2000,1000]

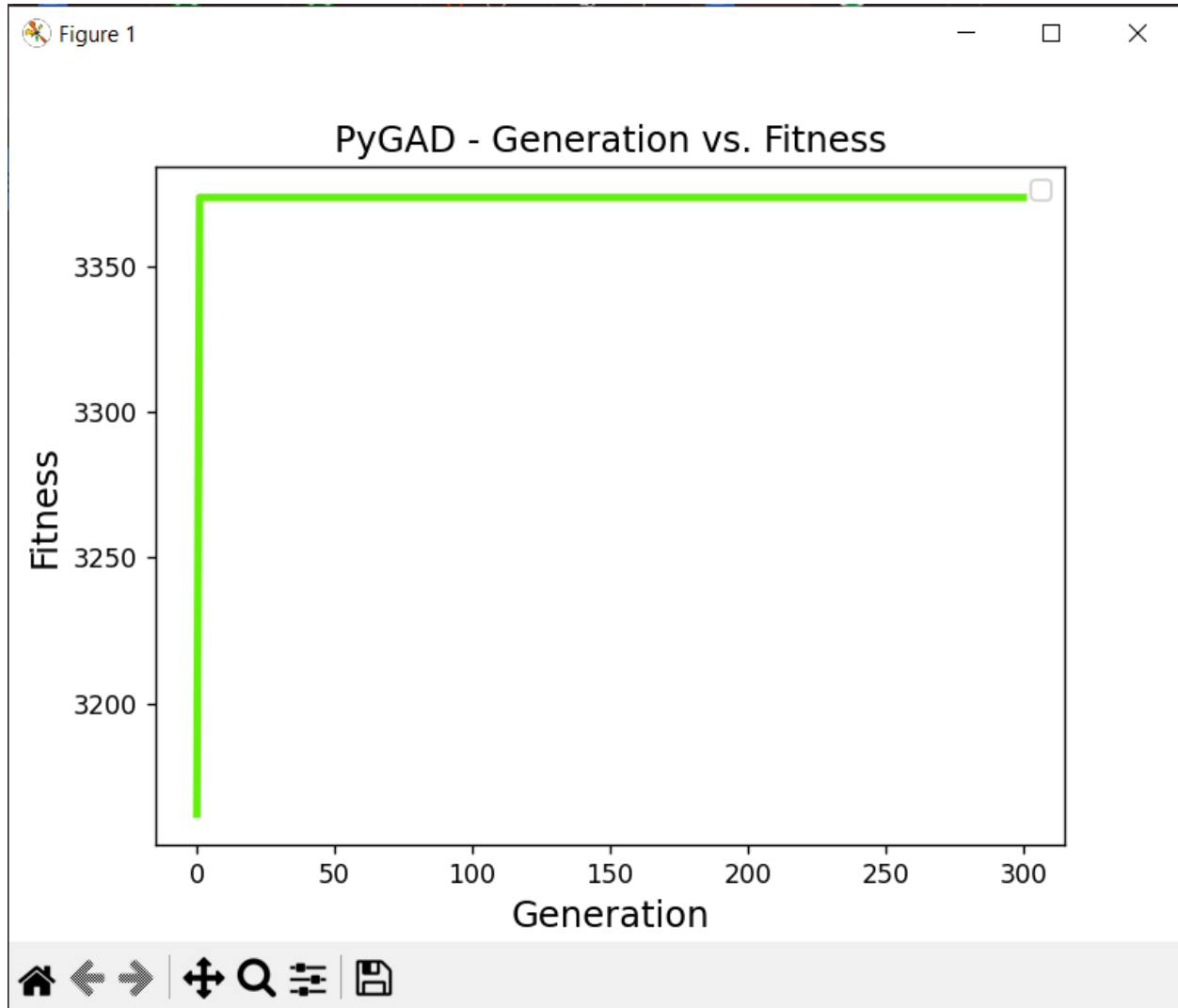
parent_selection_type = "tournament" #antes "sss"
keep_parents = 5 #antes 1

crossover_type = "two_points"

mutation_type = "random"
mutation_percent_genes = 12 #Antes 30
gene_type = int
```

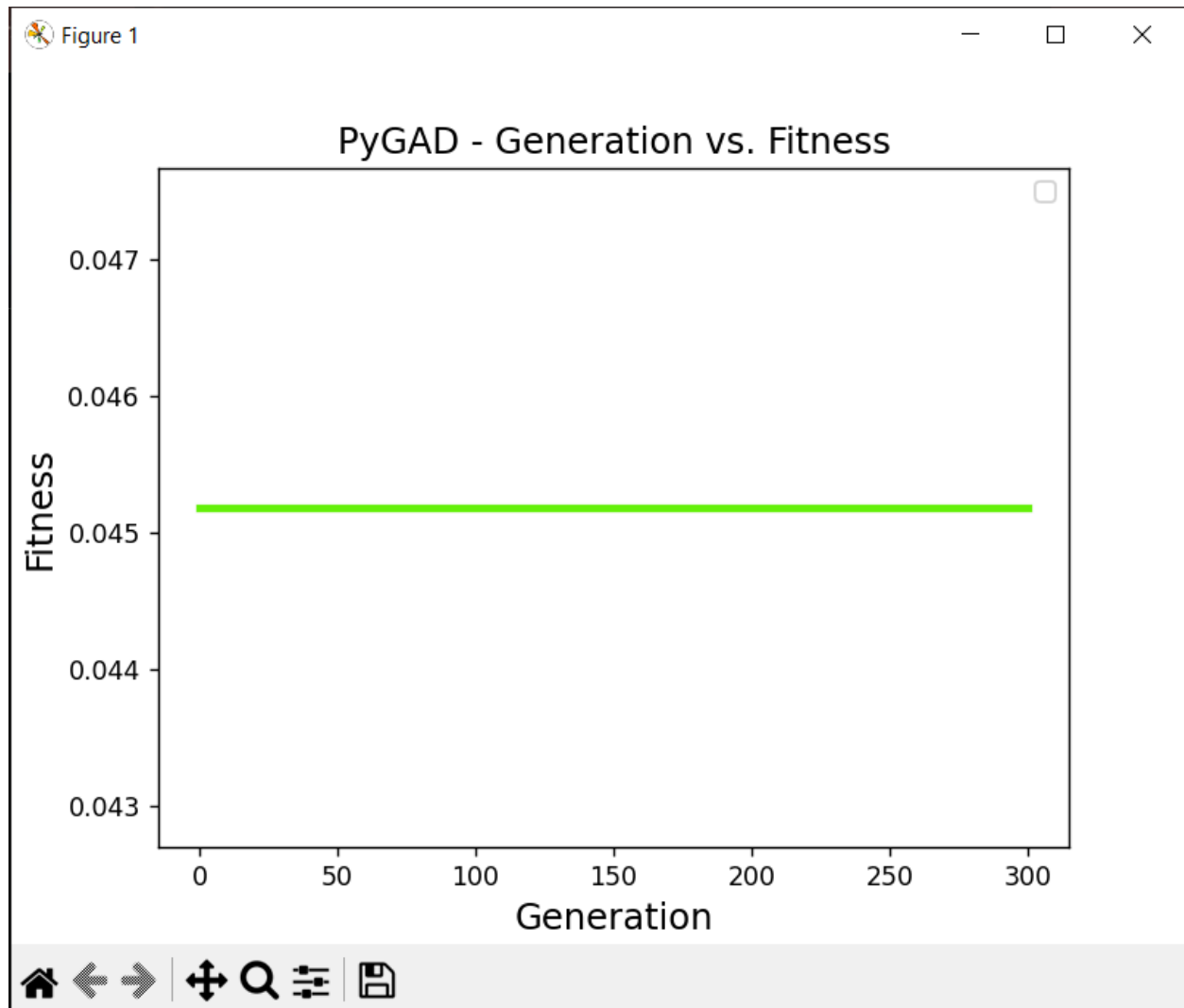
Ejecución 1

Máximo:

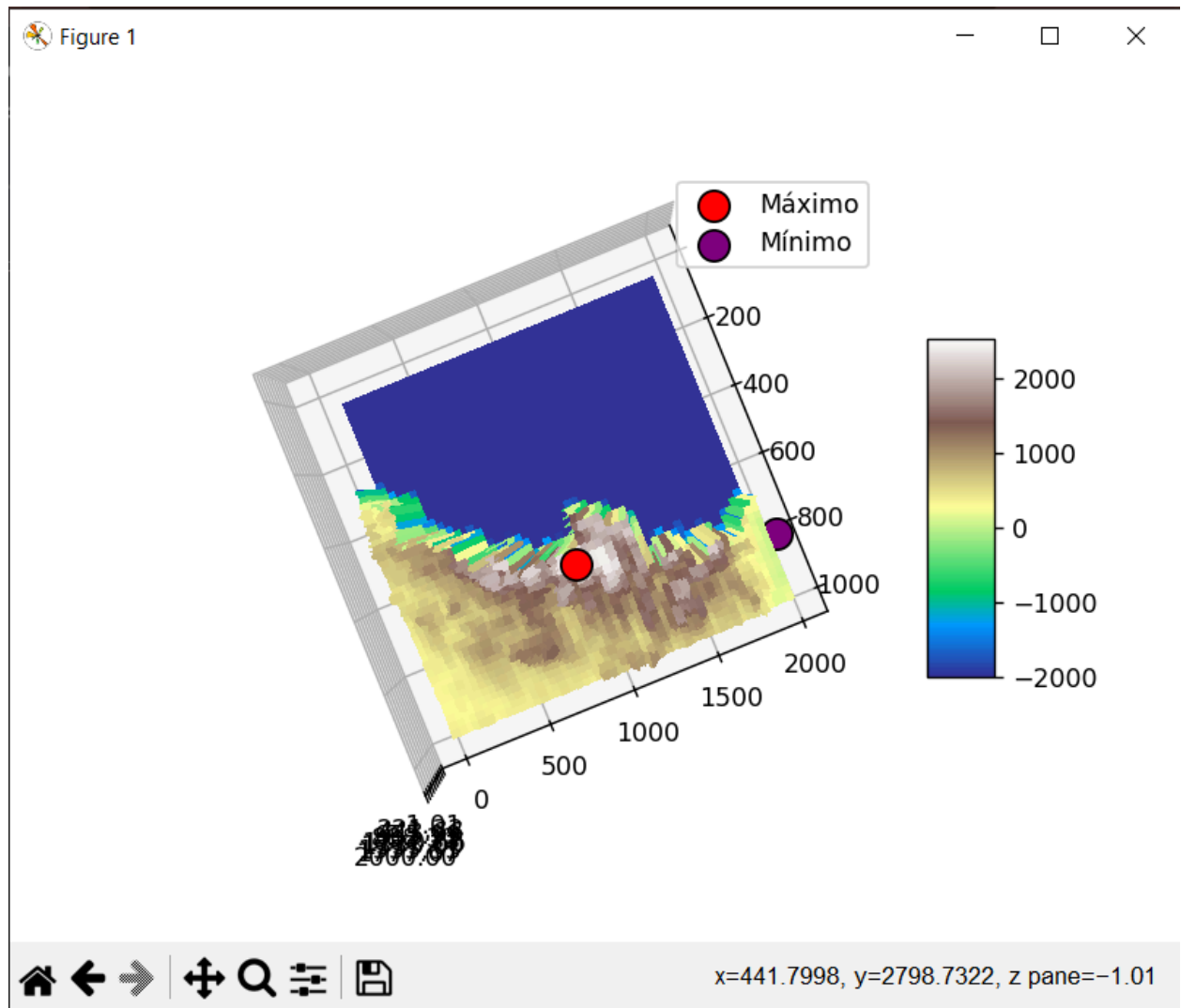


```
Parameters of the best solution : [1021 665]
Fitness value of the best solution = 3373.484
C:\Users\David\Downloads\Ficheros para la Práctica 1.1-2025
packages\pygad\visualize\plot.py:120: UserWarning: No artists
d to put in legend. Note that artists whose label start with
e ignored when legend() is called with no argument.
  matplotlib.legend()
Predicted output based on the best solution : 3373.484
Valor de altitud maximo encontrado: 3373.484
```

Mínimo:

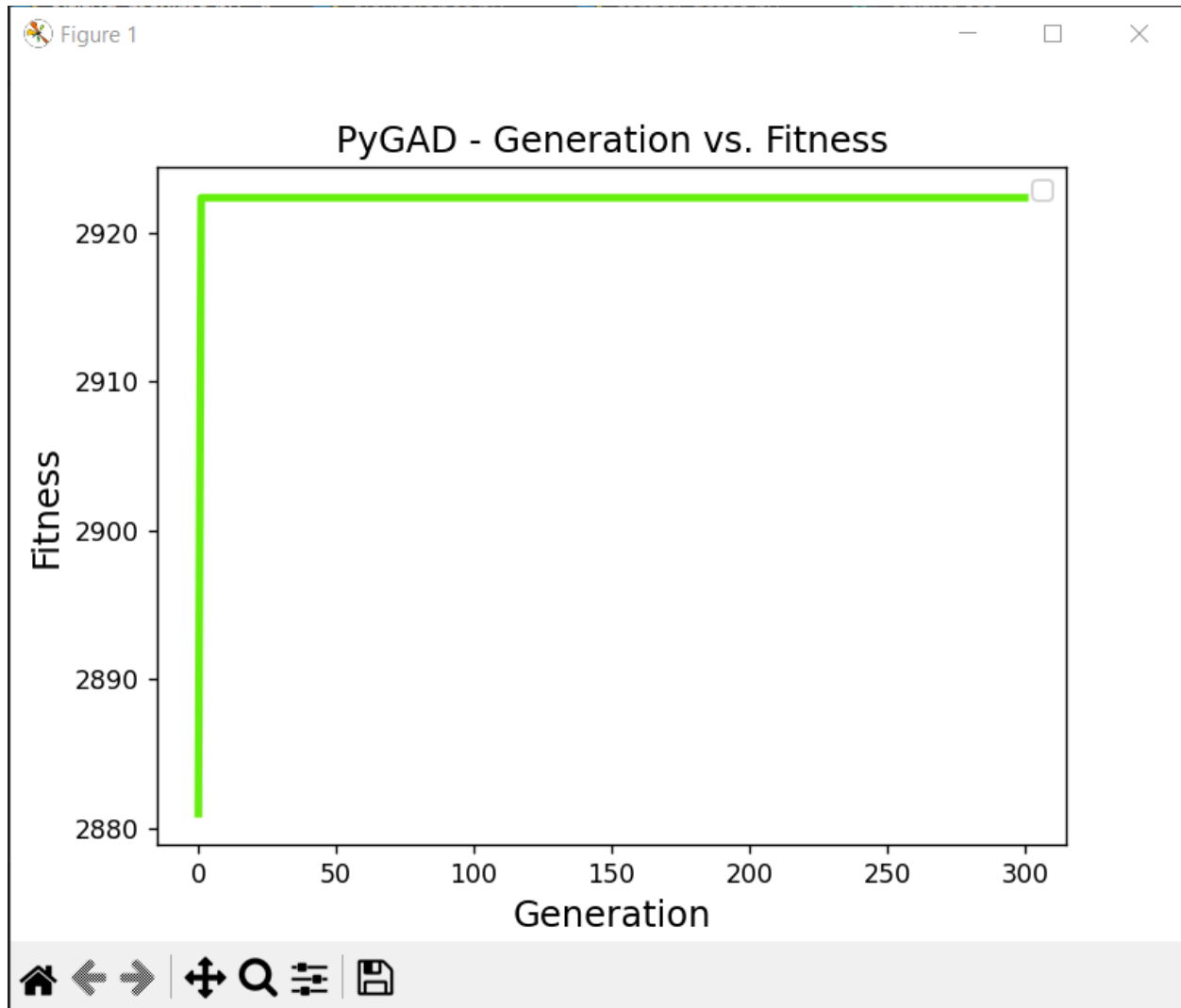


```
Parameters of the best solution : [1990  797]
Fitness value of the best solution = 0.04518140129302845
Predicted output based on the best solution : 22.133
Valor de altitud minimo encontrado: 22.133
```



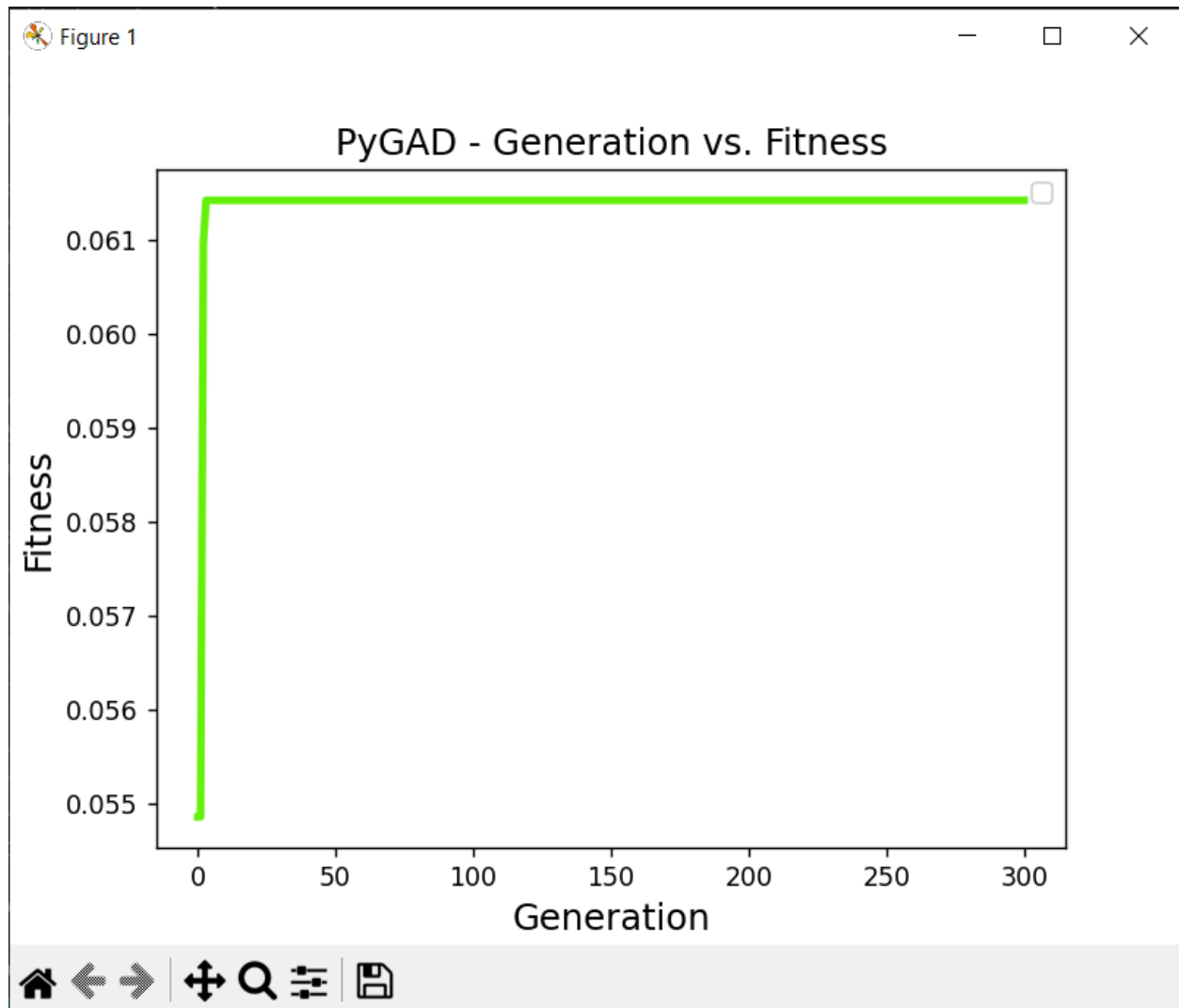
Ejecución 2

Máximo:

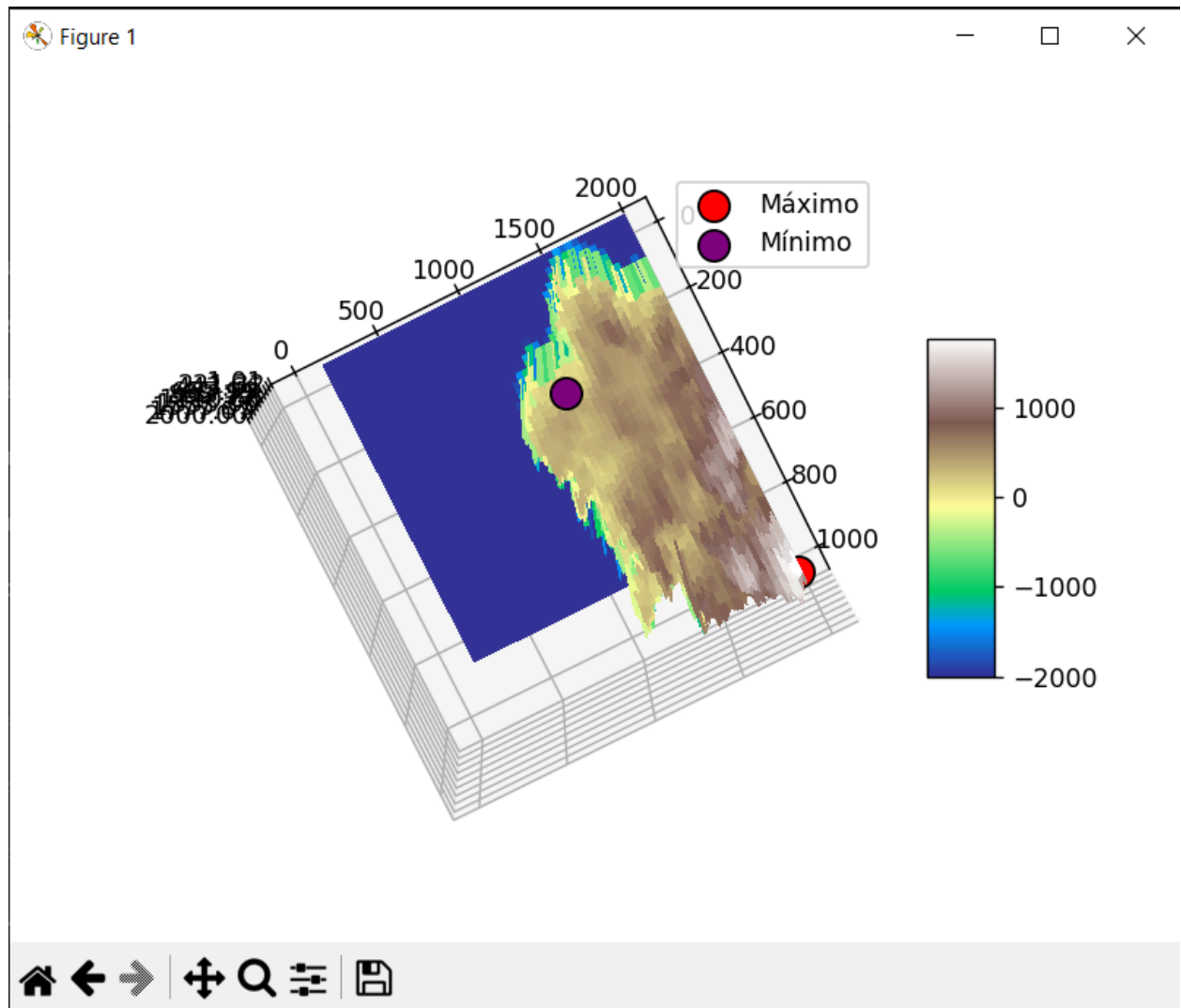


```
Parameters of the best solution : [642 601]
Fitness value of the best solution = 2922.35
C:\Users\David\Downloads\Ficheros para la Práctica 1.1-20251104\venv\li
t artists whose label start with an underscore are ignored when legend(
    matplotlib.legend()
Predicted output based on the best solution : 2922.35
Valor de altitud maximo encontrado: 2922.35
```


Mínimo:

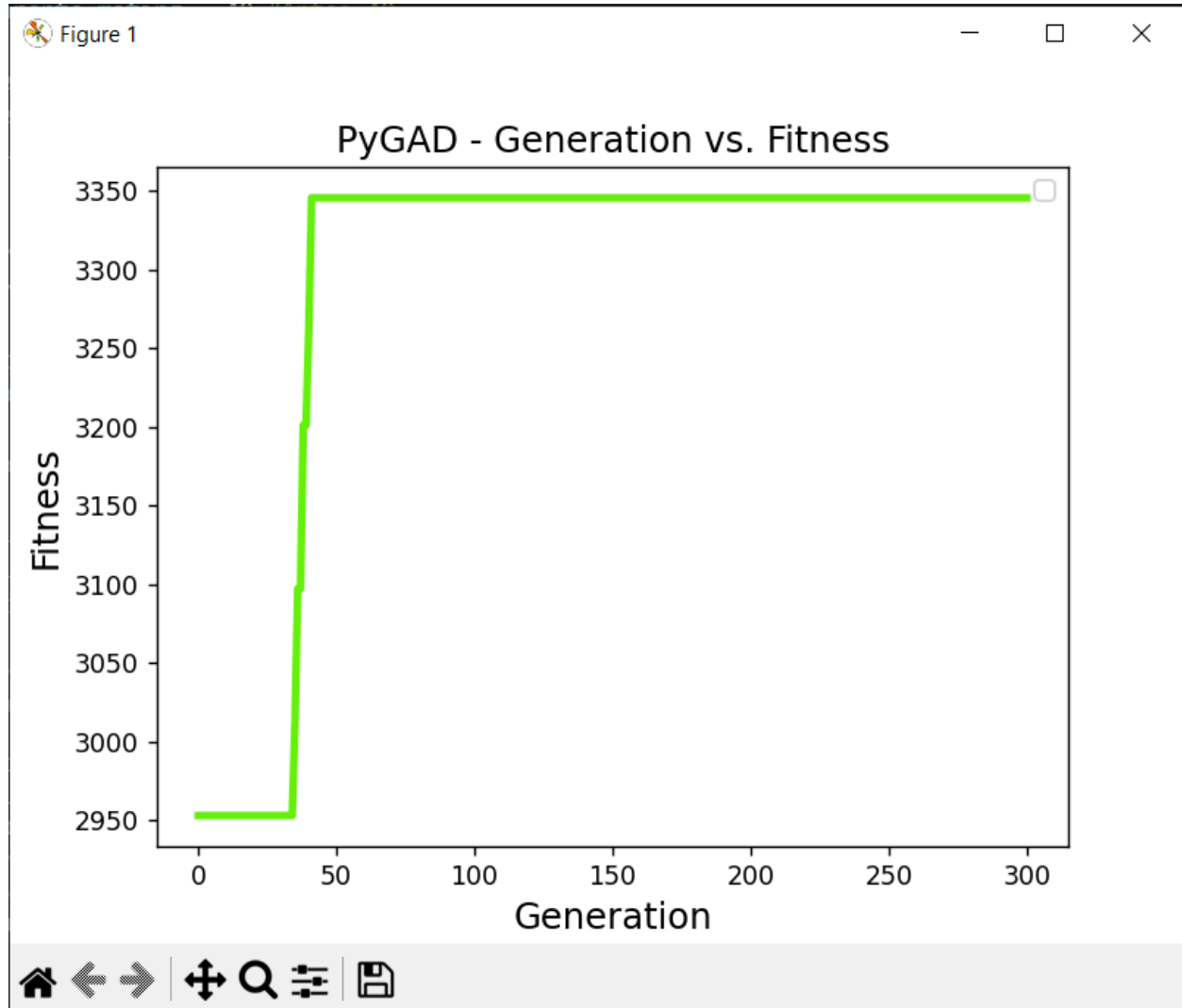


```
=====
Parameters of the best solution : [1992  807]
Fitness value of the best solution = 0.0607496469989887
Predicted output based on the best solution : 16.461
Valor de altitud minimo encontrado: 16.461
```



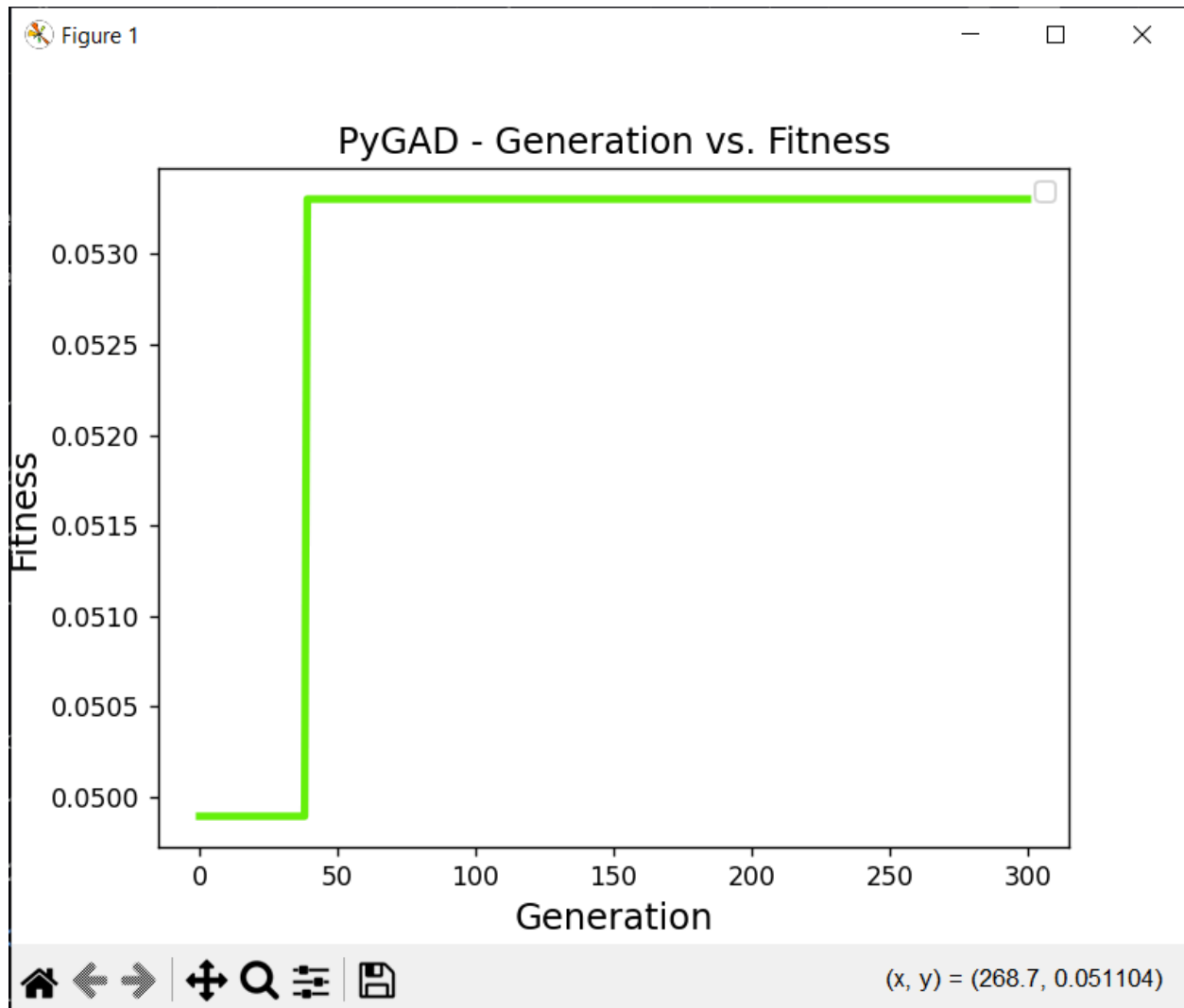
Ejecución 3

Máximo:

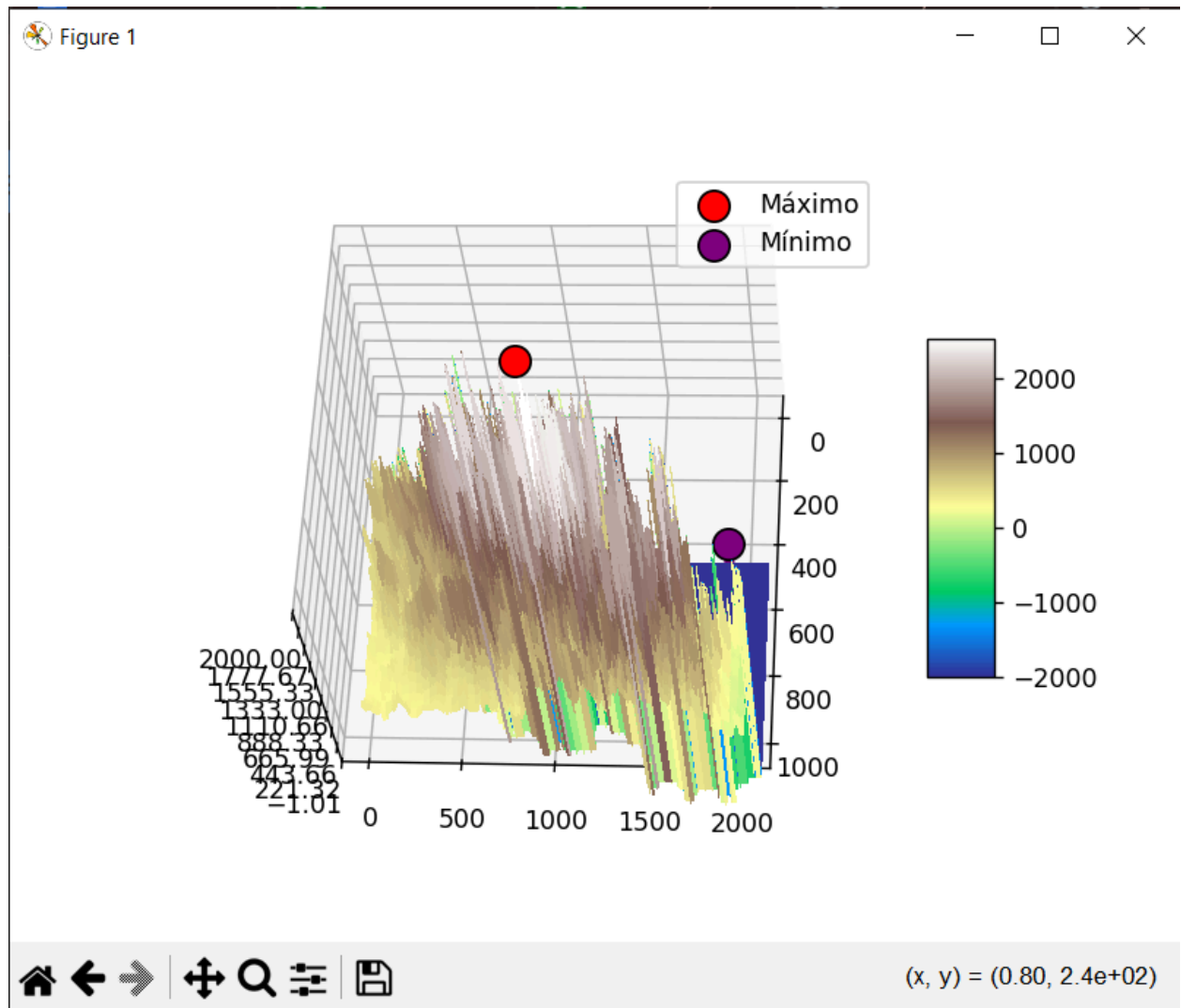


```
Parameters of the best solution : [1017  838]
Fitness value of the best solution = 3223.69
C:\Users\David\Downloads\Ficheros para la Práctica 1.1-20251104\venv\lib\site-
t artists whose label start with an underscore are ignored when legend() is ca
matplotlib.legend()
Predicted output based on the best solution : 3223.69
Valor de altitud maximo encontrado: 3223.69
```

Mínimo:



```
=====
Parameters of the best solution : [1992  868]
Fitness value of the best solution = 0.05238344408677611
Predicted output based on the best solution : 18.761
Valor de altitud minimo encontrado: 18.761
```



Conclusión

Los extremos de altitud siguen siendo claramente identificables: los máximos se sitúan alrededor de 3200 m, mientras que los mínimos permanecen cerca del nivel del mar, con valores próximos a 15 m. Comparado con los ajustes anteriores, los máximos encontrados son ligeramente más conservadores, lo que indica que el algoritmo ha explorado mejor el mapa y ha evitado quedarse en máximos locales extremadamente aislados. La selección por rango y la mayor cantidad de generaciones han permitido una búsqueda más exhaustiva, manteniendo la gran diferencia altitudinal entre los picos y los valles, y reflejando un relieve heterogéneo con contrastes pronunciados. En general, el algoritmo ajustado muestra estabilidad y coherencia en la localización de los puntos extremos, con valores más consistentes entre muestras.